MSML 605 Milestone 2 Evaluation Report

Team : Rahul Chithra Shiva
&
Anantha Aksshaj
Erigisetty

1. Introduction

This report covers Milestone 2 of the MSML 605 face verification project. The goal was to build a complete evaluation loop scoring face pairs, sweeping thresholds, tracking runs, and testing whether a data centric improvement will change the model's performance. All experiments use the LFW dataset with a pixel level cosine similarity baseline.

2. System Overview

- **Dataset:** LFW (Labeled Faces in the Wild), split 70% train / 15% val / 15% test
- **Pairs per split:** 1200 (600 same-person positives, 600 different-person negatives)
- **Feature representation:** Flattened 50×50 grayscale pixel vector (2500 dimensions)
- **Similarity metric:** Cosine similarity (range: −1 to 1, higher = more similar)
- **Threshold selection rule:** Maximize balanced accuracy on val split only
- **Threshold range swept:** −1.0 to 1.0 in 200 steps

3. Pipeline Modules

src/validation.py -> Validates pairs, scores, thresholds, config, and split leakage
src/evaluation.py -> Loads images, score pairs, applies threshold
src/metrics.py ->Confusion matrix, compute_metrics, threshold_sweep, select_threshold, roc_data
src/tracking.py -> Logs each run to JSON
scripts/evaluate.py -> End-to-end CLI: load → score → sweep/apply → plot → log
scripts/create_pairs_v2.py -> Generates filtered pairs (data-centric experiment)

tests/ -> 47 unit and integration tests all have passing

4. Experiments & Results

Baseline (runs 1 to 3): Original pairs, no filtering.

Data-centric (runs 4 and run 5):
For the data-centric change, we looked at the identity distribution in the LFW dataset and noticed that some identities only had a single image, meaning it was impossible to ever form a positive pair from them. We dropped those identities entirely since they could only contribute negative pairs. We also noticed a couple of very highly repetitive identities with dozens of images dominating the dataset, so we kept a limit of 10 images to prevent any one person from being overrepresented. After applying these two filters, we regenerated the pairs into outputs/pairs_v2/ and re-ran the sweep and test evaluation as runs 4 and 5.

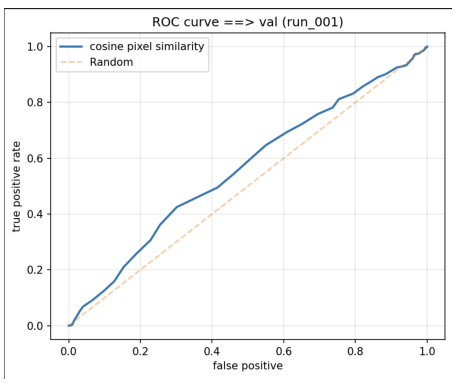
Run ID	Split	Data	Threshold	Balance Accuracy	F1 score	TPR	FPR	Precision
Run 1	val	baseline	0.7789	0.5617	0.4923	0.425	0.3017	0.5849

Run 2	val	baseline	0.7789	0.5617	0.4923	0.425	0.3017	0.5849
Run 3	test	baseline	0.7789	0.5958	**0.548**	0.490	0.2983	0.6216
Run 4	val	filtered	0.7387	0.5867	0.5967	0.6117	0.4383	0.5825
Run 5	test	filtered	0.7387	0.5558	0.5726	0.595	0.4833	0.5518

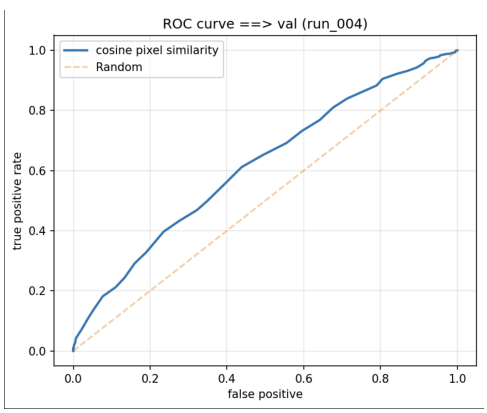
Run_001 and run_002 are similar. They confirm that locking a swept threshold and re-running produces the same result, which confirms the pipeline's reproducibility.

5. ROC CURVE:

To generate the ROC curves, we swept the cosine similarity threshold across 200 evenly spaced values between −1.0 and 1.0. at each threshold, we recorded the TPR and FPR, then plotted them against each other. Ideally we want the curve to bend toward the top-left corner, which would mean the model is catching most of the true matches while keeping false alarms low. as you can see in the plots, our curves sit pretty close to that diagonal, which confirms that raw pixel similarity is not a strong signal for telling people apart.



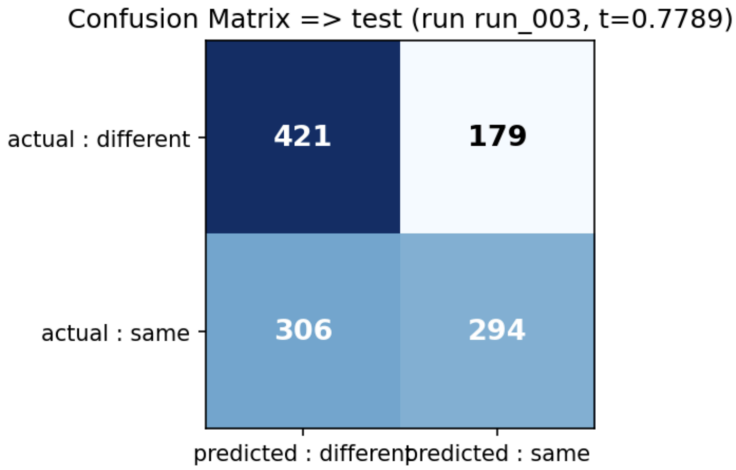
(ROC curve from run 1)



(ROC curve from run 4)

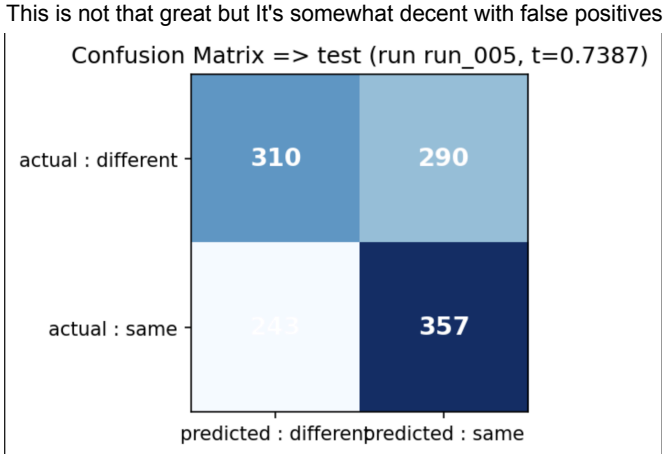
6. Confusion matrix:

We pulled the confusion matrices from the two test set runs since those are the final numbers that actually matter. val was only used to pick the threshold.



Looking at the baseline for run_003, t = 0.778894, the thing that jumps out immediately is the false negatives, which is 306 out of 600 same person pairs that were missed. That means the model is getting more than half of the genuine matches wrong,

This is not that great but It's somewhat decent with false positives which is 179, It is more often right than wrong



When we look at the filtered run_005, t = 0.7387, the story flips a bit. The lower threshold means the model is more willing to call pairs a match, so true positives went up from 294 to 357, which sounds good. But false positives jumped from 179 to 290 at the same time, which cancels out the gain. The end result is a worse balanced accuracy on the test set (0.5558 vs 0.5958), which tells us that lowering the threshold just changed one type of error for another rather than actually improving the model's ability to discriminate.

7. Error Slice Analysis

We were not able to inspect individual image pairs directly during this milestone, so the following slices are based on reasoning about the feature's known limitations rather than manual image review. That said, the confusion matrix counts give us a clear enough picture of where the model is struggling.

Slice A => False Negatives (FN = 306 / 600 -> 51%)

The model missed more than half of the actual same-person pairs. The most likely reason we think is that pixel vectors are extremely sensitive to lighting and pose changes. If the same person is photographed inside in one image and outside in another, the brightness values shift completely, even though it's the same face. The same thing happens with expressions or accessories like glasses , the pixel pattern changes enough that the cosine similarity drops below the threshold and the model calls it a mismatch. There is no way for this feature to know "same face, different conditions."

Slice B => False Positives (FP = 179 / 600 ->wh 29.8%)

About 30% of different-person pairs were incorrectly called a match. This makes sense when you think about what cosine similarity is actually measuring , it is comparing brightness patterns across a 50×50 grid, not facial identity. Two completely different people photographed in the same studio with similar lighting and skin tone can end up with very similar pixel vectors. At that resolution, the fine details that actually distinguish faces such as eye shape or nose structure basically disappear.

We think that both the slices point to the same root problem, the feature itself is too weak for this task. No amount of tuning the threshold or data cleaning will fix this.

8. Conclusion

The data filtering did not actually improve test performance. The balanced accuracy actually dropped from 0.5958 to 0.5558, which tells us that the problem is not the data, it is the feature itself. Raw pixel similarity has no way of handling the kind of variation that shows up in real face images. The natural next step would be replacing the pixel vector with a learned face embedding from a pretrained CNN, which would directly address both failure slices identified above.